Tarea 8

Jardón Marín Rodrigo

Investigar:

Tipos de datos en PostgreSQL

Numéricos: Los tipos de datos numéricos en PostgreSQL abarcan distintas capacidades de almacenamiento en bytes y niveles de precisión, adaptándose a las necesidades específicas de cada valor. La enumera todos los tipos disponibles dentro de esta categoría.

Nombre	Tamaño de almacenamiento	Descripción	Rango
`pequeñoint`	2 bytes	entero de rango pequeño	-32768 a +32767
`entero`	4 bytes	elección habitual para entero	-2147483648 a +2147483647
`bigint`	8 bytes	entero de amplio rango	-9223372036854775808 a 9223372036854775807
`decimal`	variable	precisión especificada por el usuario, exacta	sin límite
`numérico`	variable	precisión especificada por el usuario, exacta	sin límite
`real`	4 bytes	precisión variable, inexacta	precisión de 6 dígitos decimales
`doble precisión`	8 bytes	precisión variable, inexacta	precisión de 15 dígitos decimales
`de serie`	4 bytes	entero autoincrementable	1 a 2147483647
`gran serie`	8 bytes	entero autoincrementable grande	1 a 9223372036854775807

- Enteros: Tipos de Enteros

Esta categoría incluye los tipos smallint, integer y bigint, los cuales almacenan exclusivamente números enteros (sin decimales). La elección del tipo adecuado depende de las necesidades de almacenamiento y del rango numérico requerido:

integer (4 bytes): Es la opción estándar y la más recomendada. Ofrece el mejor equilibrio entre capacidad, uso de memoria y rendimiento en el procesamiento.

smallint (2 bytes): Su uso se recomienda únicamente en escenarios donde la optimización del espacio en disco es crítica.

bigint (8 bytes): Debe reservarse para casos específicos donde el rango de valores supere la capacidad del tipo integer, ya que su tamaño es mayor y puede ser procesado un poco más lento.

- Precisión Arbitraria:

El tipo numérico (totalmente equivalente a decimal en el estándar SQL) está diseñado para realizar cálculos exactos. Puede almacenar números con hasta 1000 dígitos de precisión, lo que lo convierte en la elección obligatoria para datos monetarios, contables o cualquier cantidad que exija exactitud absoluta. Como contraparte, las operaciones matemáticas con este tipo de datos son más lentas en comparación con los enteros.

Para definir cómo se almacena un valor numérico, se utilizan dos parámetros fundamentales:

Precisión: Es la cantidad total de dígitos significativos en todo el número (contando ambos lados del punto decimal).

Escala: Es la cantidad de dígitos que se ubican en la parte fraccionaria (a la derecha del punto decimal).

Al crear una columna, este tipo de dato se puede configurar de tres formas:

NUMERIC (precisión, escala): Establece los límites exactos definidos por el usuario.

NUMERIC (precisión): Asigna la precisión especificada y establece una escala de cero por defecto (comportándose funcionalmente como un entero).

NUMERIC (sin parámetros): Crea una columna flexible capaz de almacenar valores de cualquier precisión y escala, hasta el límite máximo permitido por la implementación del motor de base de datos.

Además, el tipo numérico es capaz de registrar el valor especial NaN (Not a Number), utilizado para representar resultados indefinidos.

- Punto Flotante:

Los tipos real y doble precisión (double precision) están diseñados para almacenar números de precisión variable. Su característica principal es que son inexactos; en la práctica, almacenan aproximaciones de los valores basadas en el estándar IEEE 754 de aritmética binaria.

Debido a esta naturaleza inexacta, al guardar o imprimir ciertos valores pueden surgir ligeras variaciones. Por esto, es crucial tener en cuenta lo siguiente:

Restricción de uso: Nunca deben utilizarse para cálculos que exijan exactitud absoluta (como registros financieros o monetarios). Para esos casos, se debe utilizar el tipo numérico.

Comparaciones: Evaluar si dos valores de punto flotante son "exactamente iguales" puede arrojar resultados inesperados debido a discrepancias microscópicas en el redondeo.

Capacidades de almacenamiento:

real (Precisión simple): Maneja un rango aproximado de 1E-37 a 1E+37, garantizando una precisión de al menos 6 dígitos decimales.

doble precisión (Precisión doble): Amplía el rango de 1E-307 a 1E+308, ofreciendo una precisión de al menos 15 dígitos.

Valores Especiales:

Al igual que otros sistemas, estos tipos soportan estados especiales que escapan de la numeración ordinaria. Al ingresarlos en comandos SQL, deben escribirse entre comillas simples (por ejemplo: UPDATE tabla SET valor = 'Infinity'):

Infinity (Infinito positivo)

-Infinity (Infinito negativo)

NaN (Not a Number - No es un número)

Notación Alternativa (Estándar SQL):

PostgreSQL también permite declarar estas columnas usando la sintaxis estándar float(p), donde p indica la precisión mínima en bits:

Declarar de float(1) a float(24) asignará internamente el tipo real.

Declarar de float(25) a float(53) asignará internamente el tipo doble precisión.

- Serie: Los tipos serial y bigserial no son tipos de datos reales, sino un atajo de PostgreSQL para crear columnas numéricas que se incrementan automáticamente al vincularles un contador interno con la regla NOT NULL. Es fundamental tener en cuenta que ser autoincrementable no convierte a la columna en llave primaria; esto debe definirse explícitamente con la restricción PRIMARY KEY. Para insertar nuevos registros, simplemente se omite esta columna o se usa la palabra DEFAULT para que el sistema genere el siguiente número. En la práctica, se utiliza serial (basado en un integer estándar) para la gran mayoría de los casos, reservando bigserial (basado en bigint) únicamente para tablas que superarán los miles de millones de registros. Como detalle final, si alguna vez se elimina esta columna, su contador interno se borrará automáticamente junto con ella.

Carácter:

PostgreSQL ofrece tres opciones principales para manejar cadenas de texto: `varchar(n)`, `char(n)` y `text`. Los tipos `varchar(n)` y `char(n)` almacenan cadenas con un límite máximo de *n* caracteres; si se intenta guardar un texto que exceda este límite, el sistema generará un error. La diferencia fundamental entre ambos es que `varchar(n)` guarda únicamente los caracteres introducidos (longitud variable), mientras que `char(n)` mantiene una longitud fija, rellenando automáticamente con espacios en blanco el espacio sobrante si la cadena es más corta (aunque el sistema ignora estos espacios al comparar valores). Por su parte, el tipo `text` permite almacenar cadenas de cualquier longitud sin necesidad de definir un límite. En cuanto a su almacenamiento, todos ocupan 4 bytes adicionales al tamaño del texto real, permitiendo guardar textos muy extensos de hasta 1 GB de capacidad máxima, los cuales el sistema se encarga de comprimir en segundo plano para optimizar el disco.

Fecha:

PostgreSQL ofrece tipos de datos específicos para manejar cualquier combinación de fechas y horas. De forma muy resumida, los principales son:

date: Guarda exclusivamente la fecha del calendario (año, mes, día), sin incluir horas.

time: Guarda exclusivamente la hora del día (horas, minutos, segundos, microsegundos). Puede configurarse para incluir o ignorar la zona horaria.

timestamp: Es la combinación de los dos anteriores (guarda fecha y hora juntas). Es el tipo más utilizado en bases de datos. Al igual que el anterior, tiene dos versiones: con zona horaria (timestamptz) o sin ella.

interval: No guarda una fecha específica, sino una duración o lapso de tiempo (por ejemplo: "3 años, 2 meses y 5 días").

Tipo Booleano

El tipo de dato boolean en PostgreSQL ocupa 1 byte de almacenamiento y está diseñado para manejar una lógica de tres estados: verdadero (TRUE), falso (FALSE) y un estado "desconocido" representado por el valor NULL. El sistema es bastante flexible al momento de ingresar estos datos, ya que ignora espacios en blanco y no distingue entre mayúsculas y minúsculas. Para registrar un estado verdadero, la base de datos acepta entradas de texto como true, yes, on o el número 1 (incluso reconoce prefijos únicos como la letra t). De manera equivalente, para registrar un estado falso, reconoce cadenas como false, no, off o el número 0 (junto con prefijos como la letra f)

Tipos Monetarios (Money)

El tipo de dato money ocupa 8 bytes de almacenamiento y está diseñado específicamente para guardar cantidades de dinero con una precisión fraccionaria fija. Una característica muy importante es que su formato (como el símbolo de la moneda) y la cantidad de decimales dependen completamente de la configuración regional de la base de datos (el parámetro lc_monetary). Por esto, hay que tener mucho cuidado al exportar e importar estos datos entre servidores de distintos países.

Aunque el sistema es flexible y permite convertir valores desde otros tipos de datos, la regla de oro aquí es nunca usar números de punto flotante (como real o doble precisión) para el dinero debido a los inevitables errores de redondeo. Si necesitas hacer operaciones matemáticas complejas (como divisiones precisas sin perder centavos), la propia documentación recomienda convertir temporalmente el valor money al tipo numérico (que vimos antes), hacer el cálculo, y luego regresarlo a money